



AsyncGen: An Asynchronous Execution and Distributed Multi-Agent Conversation Framework Integrating AutoGen and Ray

Student: Zhen-Jing Xie

Advisor: Prof. Chia-Yu Lin, Prof. Ted T. Kuo

Outline



- Introduction
- Related Works
- Method
- Experiments
- Conclusion
- Limitation & Future Work

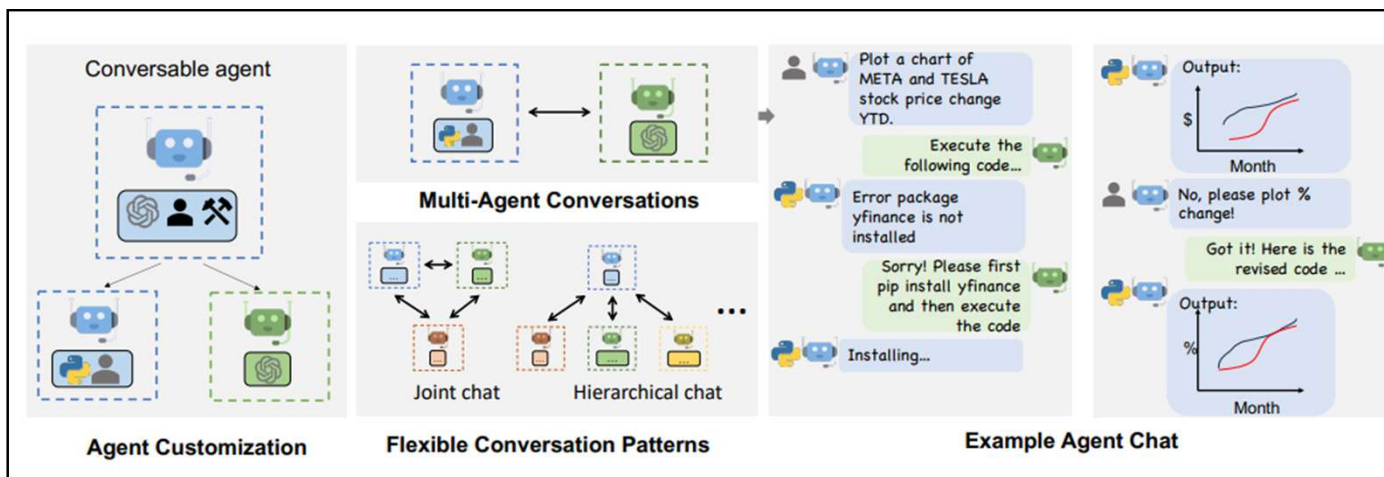


Background

- Recently, **large language models (LLMs)** have become key to developing powerful agents that leverage LLMs for reasoning and tool use to accomplish many real-world tasks.
- In order to enable LLMs to support more complex tasks, an intuitive approach is to have **multiple agents collaborate with each other**.
- To achieve this goal, Microsoft developed a multi-agent conversation framework called **AutoGen**, which allows users to easily and quickly use different LLM models for conversation.
- However, **AutoGen runs all agents sequentially** within a single process and a single thread, resulting in performance bottlenecks and insufficient hardware resources.
- Addressing this limitation is crucial to enabling the framework to handle more complex and compute-intensive tasks efficiently.

AutoGen

- Title: AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation
- Challenge:
 - How to design individual agents that are capable, reusable, customizable, and effective in multi-agent collaboration.
 - How to develop a straightforward, unified interface that can accommodate a wide range of agent conversation patterns
- Approach:
 - Customizable and conversable agents: Flexible agents combining LLMs, tools, and human input for diverse roles.
 - Conversation programming: Simplifies workflows by defining agent roles and interactions using natural and programming languages.
- Limitation: All agents run on a single thread and a single machine.



WU, Qingyun, et al. Autogen: Enabling next-gen llm applications via multi-agent conversation framework. *arXiv preprint arXiv:2308.08155*, 2023.



Motivation

Currently, the issue with AutoGen is that all agents run within a single thread and on the same machine, which leads to three main problems:

1. One major issue is that if an agent requires a long time to execute, other **agents are forced to queue up**, leading to performance bottlenecks.
2. Additionally, when certain agents rely on specific hardware resources, implementing them becomes challenging since **all agents are constrained to run on the same machine**.
3. Furthermore, the single-threaded nature of the system means that **a failure in any single agent can cause the entire system to crash**, emphasizing its critical lack of fault tolerance.

To address the above issues, we use **Ray**. Ray enables multiple agents to run in different processes or even on different machines, thereby enhancing execution efficiency, scalability, and fault tolerance.

Goal

In this research the main objectives: Integrate **AutoGen** with **Ray**, enabling the system to have the following capabilities:

- **Asynchronous processing:** Agents can run concurrently, and the system can continue the task without waiting for everyone to complete.
- **Support for heterogeneous hardware:** The system can allocate agents to run on the appropriate machines based on their task requirements.
- **Fault tolerance:** The system can recover the state and continue the task when an agent crashes.

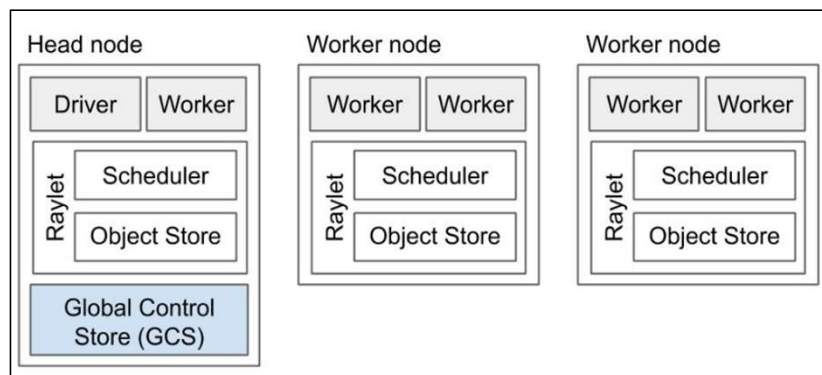


Fig. Ray Cluster Architecture

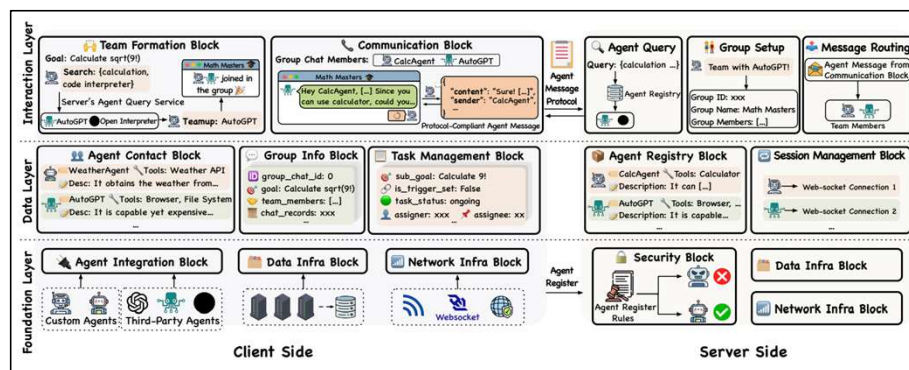
Ray v2 Architecture white paper (https://docs.google.com/document/d/1tBw9A4j62rul5omlJbMxly-la5w4q_TjyJgJL_jN2fl/preview?tab=t.0#heading=h.iym5j2gcdoq)



Related Works

Internet of Agents (IoA)

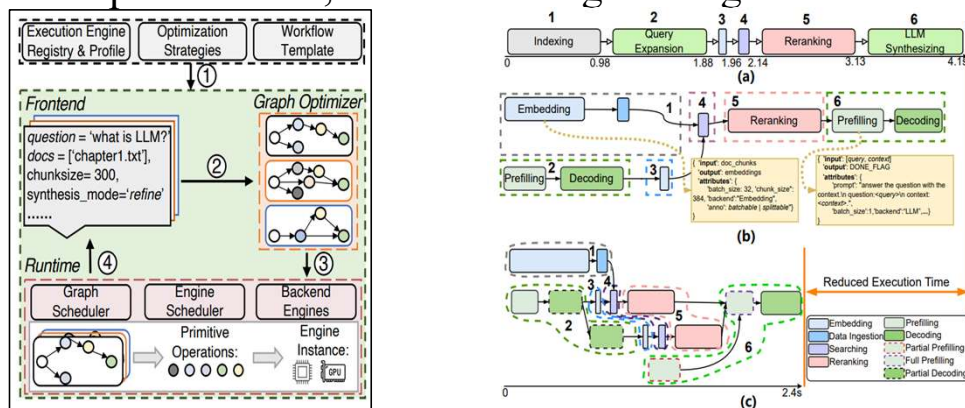
- Title: Internet of Agents: Weaving a Web of Heterogeneous Agents for Collaborative Intelligence
- Challenge:
 - **Ecosystem Isolation:** Frameworks often lack integration with third-party agents, limiting diversity and generality.
 - **Single-Device Simulation:** Agents are typically confined to a single device, unlike real-world distributed setups.
 - **Rigid Communication:** Communication and coordination are hard-coded, lacking real-world flexibility.
- Approach:
 - **Agent Registry Mechanism:** Each agent registers with the server, providing its capabilities and status for management and discovery. This enables a distributed system where heterogeneous agents, with different hardware, software, or processing methods, can collaborate seamlessly.
- Limitation: While it successfully implements a distributed system, it does not address fault tolerance within the framework.



Chen, Weize, et al. "Internet of agents: Weaving a web of heterogeneous agents for collaborative intelligence." arXiv preprint arXiv:2407.07061 (2024).

Teola

- Title: Teola: Towards End-to-End Optimization of LLM-based Applications
- Challenge:
 - Optimizing and orchestrating complex workflows in large-scale LLM-based applications, improving both performance and efficiency.
- Approach:
 - **Graph Optimizer:** Teola uses a primitive-based p-graph to find the best execution strategy and convert it into an optimized e-graph for improved performance.
 - **Distributed Scheduling and Execution:** Teola uses Ray for distributed scheduling, enabling parallel execution across machines.
- Limitation: Using Teola requires modifying backend frameworks. While this enables finer-grained orchestration and better performance, it increases engineering effort.



Tan, Xin, et al. "Teola: Towards end-to-end optimization of llm-based applications." arXiv preprint arXiv:2407.00326 (2024).



Method

Method



In this section, we will introduce the specific methods we have implemented, presented in the following order:

- Overview
- Integration of AutoGen's agent with Ray's actor
- Architecture
- Workflow
- Implementation of Supporting Features
 - Parameter Settings
 - Backup Mechanisms
 - Synchronization of historical conversation records

AutoGen

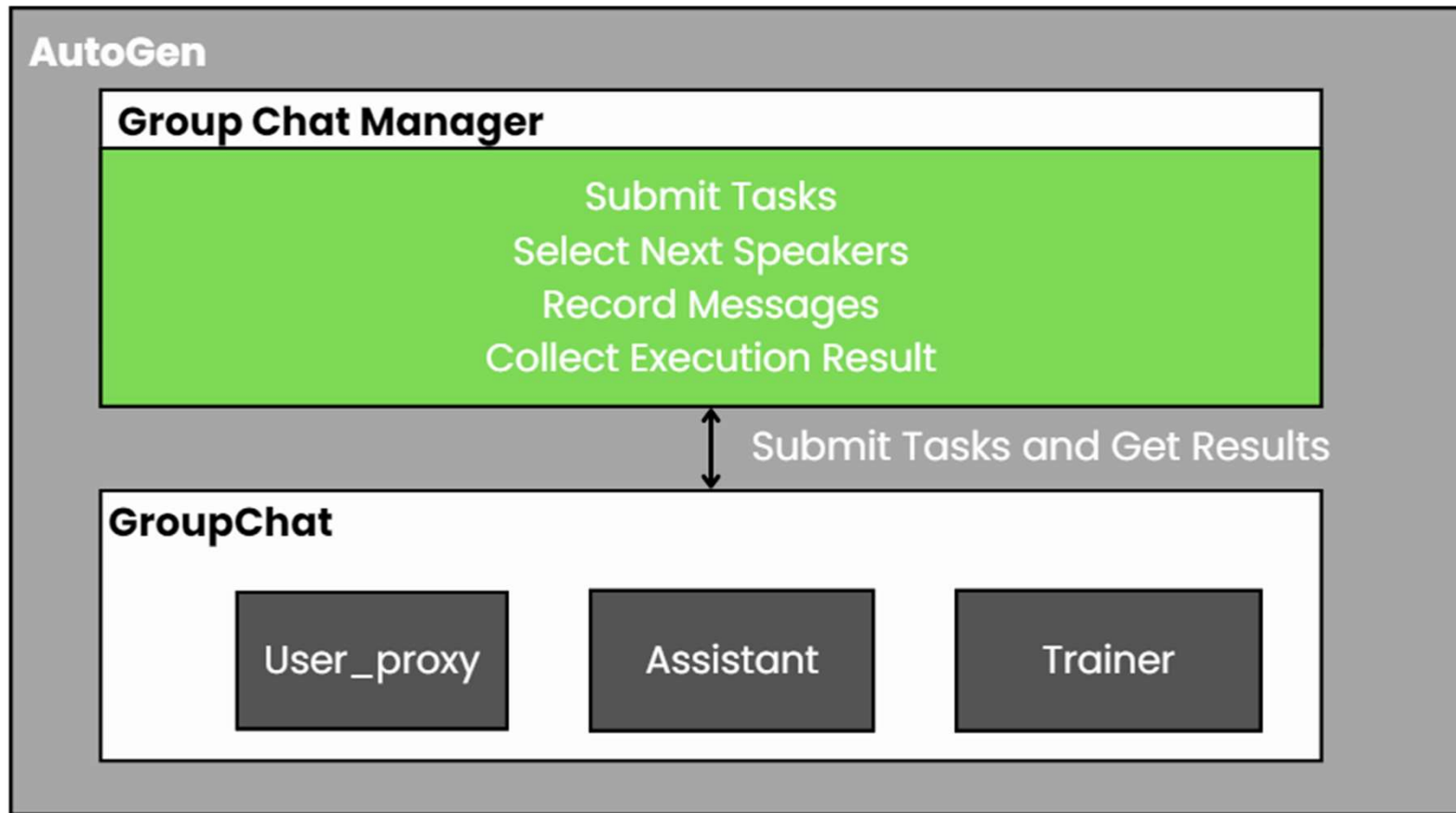


Fig. Architecture of AutoGen

AsyncGen Overview

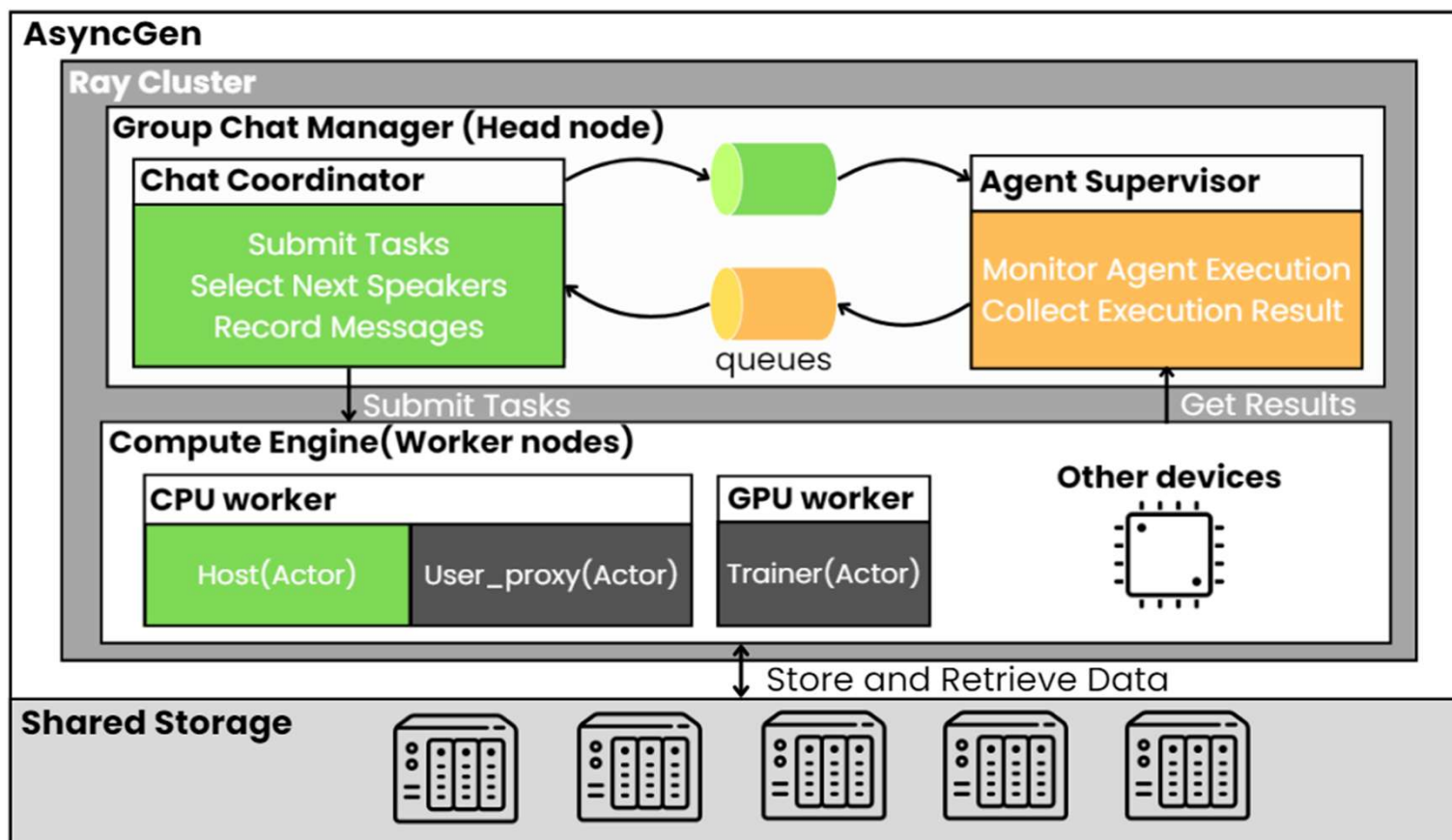


Fig. Architecture of AsyncGen



Integrate Agent and Actor

Integrate AutoGen's Agent with Ray's Actor

- AutoGen's agents are implemented as class objects, while Ray's actors are defined as classes decorated with the `@ray.remote` decorator.
- To integrate them, we can simply add the `@ray.remote` decorator to AutoGen's agents, effectively turning them into Ray actors.

`@ray.remote()`

```
class AssistantAgent(ConversableAgent):  
    code ...
```

- AutoGen's method of using agents needs to be changed to ensure compatibility:
 - Invoke the agent's methods
 - Retrieve the agent's method results
 - Access the agent's attributes

=> However, changing all the relevant syntax in the code would be **very time-consuming**, and each time AutoGen updates, we would need to revise the related code, making the direct code modification approach impractical.



Core components of Ray

Core components of Ray:

- In Ray, there are three fundamental elements: **Task**, **Actor**, and **Object**:
- **Task**:
 - A stateless worker.
 - Typically implemented in Python as a regular function with the Ray decorator **@ray.remote**.
- **Actor**:
 - A stateful worker.
 - Implemented in Python by applying the **@ray.remote** to a class.
- **Object**:
 - Used to store the execution results.
 - When a user invokes a task or an actor's method, it does not directly return the execution result but instead returns an **object reference**.

When a task is completed, the result is stored in the corresponding location within Ray's object store. Users need to **retrieve the execution result through ray.get(object reference)**. In Ray, it is crucial to carefully control the timing of calling ray.get.



Core components of Ray

Task (Remote function, stateless)

```
@ray.remote()
def func_name():
    ...
```

Object (The data stored in Ray)

```
object_ref_1 = func1_name.remote()
/// non blocking ///
object_ref_2 = func1_name.remote()
object2 = ray.get(object_ref_2)
/// blocking ///
object_ref_3 = func3_name.remote()
object1 = ray.get(object_ref_1)
object3 = ray.get(object_ref_3)
```

Actor (Remote class, stateful)

```
@ray.remote()
class cls_name():
    ...
```

.remote(): The method to instantiate and call the function in Ray.

- `func_name() => func_name.remote()`
- `cls_name() => cls_name.remote()`

.getter_func_name.remote(): The method to get the variable of actor.

- `obj.name => obj.get_name.remote()`

ray.get(): The method to get the value of object in Ray.

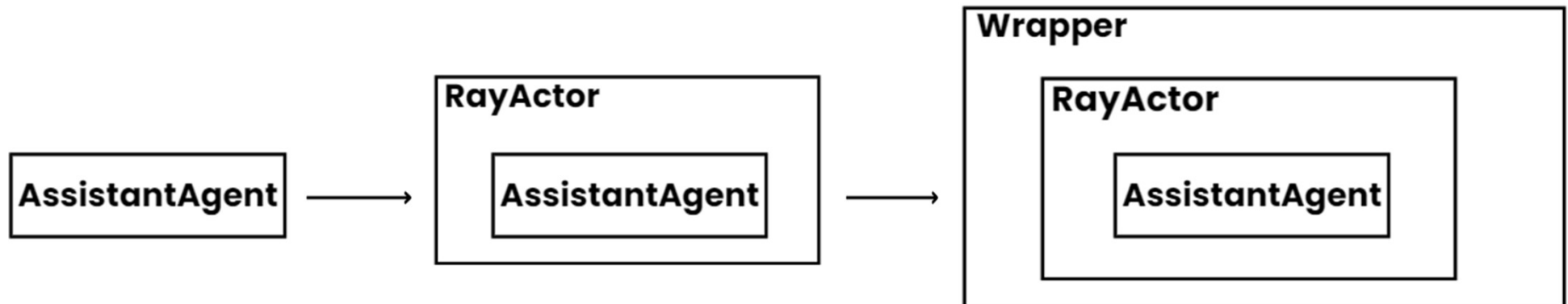
Wrapper

- We have an `agent_cls` that is transformed into an actor using `ray.remote`.
- Normally, accessing attributes of an actor (Ray remote object) is done differently compared to regular Python objects.
- **Goal:** We want to make accessing the actor's attributes feel the same as interacting with regular attributes, so users don't notice any difference.
- **Idea:** We create a **Wrapper** class to wrap the actor, acting as a bridge between the user and the actor.

Assistant.method()

Assistant.method.remote()

Assistant.method()



Wrapper Overview

- **Agent** : Agent.method()
- **Ray.remote(Agent) - actor**: Actor.method.remote()
- **Wrapper(Agent)**: Actor.method.remote()→ Wrapper.method()

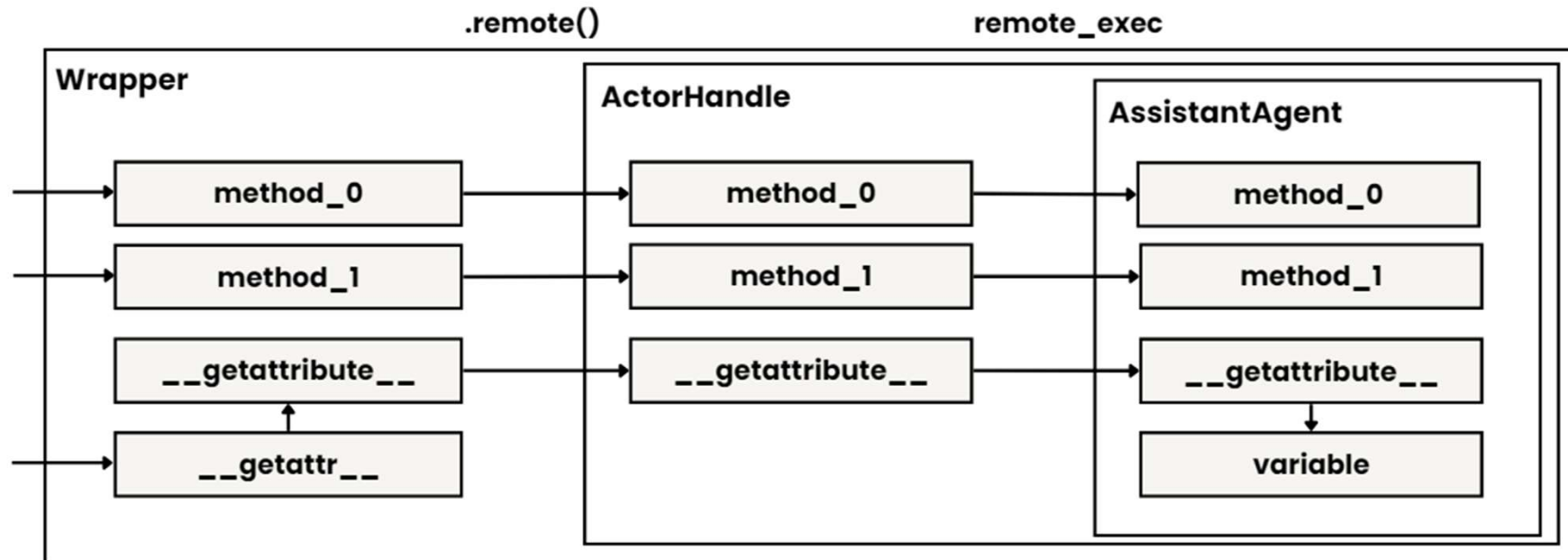


Fig. Wrapper Overview



Magic Method in Python

`__getattribute__` (*self, name*): In Python, `__getattribute__` is a magic method that is automatically **called whenever any attribute of an object is accessed**. If the code in `__getattribute__` doesn't find the attribute, it looks to see if `__getattr__` is defined. If so, it is called. Otherwise, `AttributeError` is raised.

`__getattr__` (*self, name*): In Python, `__getattr__` is a magic method that is **called when an attribute is accessed on an object but is not found** in the object's instance dictionary or its class hierarchy.

We use `__getattribute__` as the getter function for the agent to retrieve all variables within the agent, and use the `__getattr__` in wrapper to trigger the getter function.



Getter Function in Agent

- We want to use `__getattrute__` as the getter function for the agent to retrieve variables, but **Ray does not automatically convert methods that are not explicitly defined into methods that can be called using `.remote`**. Therefore, we need to manually add `__getattrute__` in each user-defined agent class.

```
8 class AssistantAgent(RayConversableAgent):
34     def __init__(
62         super().__init__(
63             name,
64             system_message,
65             is_termination_msg,
66             max_consecutive_auto_reply,
67             human_input_mode,
68             llm_config=llm_config,
69             description=description,
70             **kwargs,
71         )
72         if logging_enabled():
73             log_new_agent(self, locals())
74
75         # Update the provided description if None, and we are using the default system_message,
76         # then use the default description.
77         if description is None:
78             if system_message == self.DEFAULT_SYSTEM_MESSAGE:
79                 self.description = self.DEFAULT_DESCRIPTION
80
81         def __getattrute__(self, name):
82             return object.__getattrute__(self, name)
```

AssistantAgent

method_0

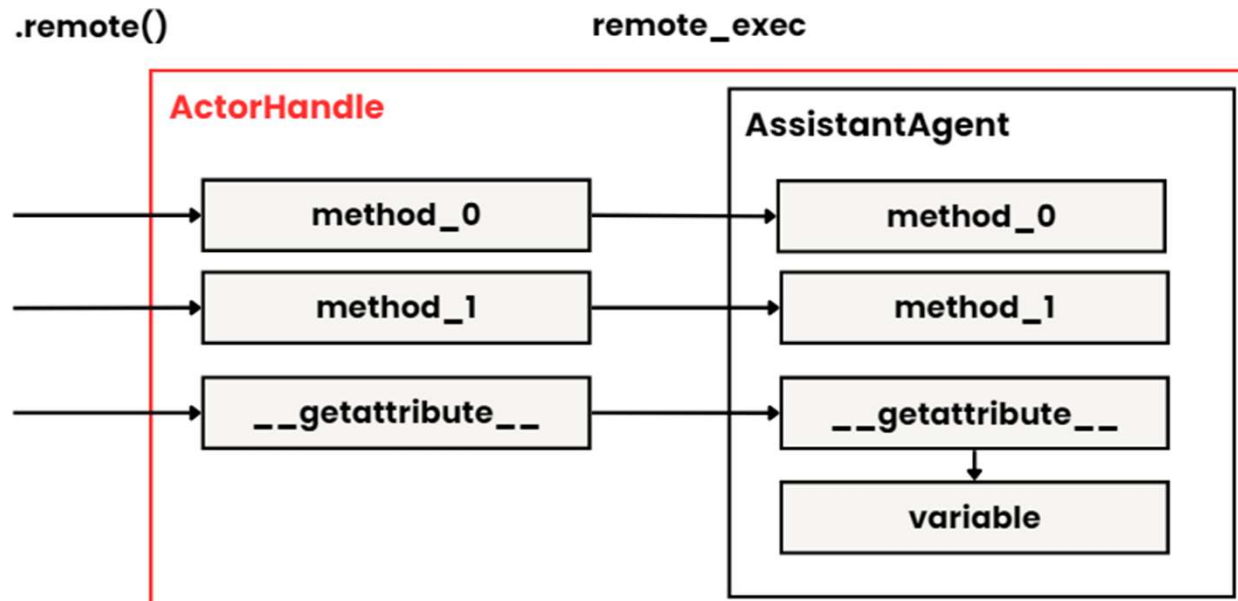
method_1

`__getattrute__`

variable

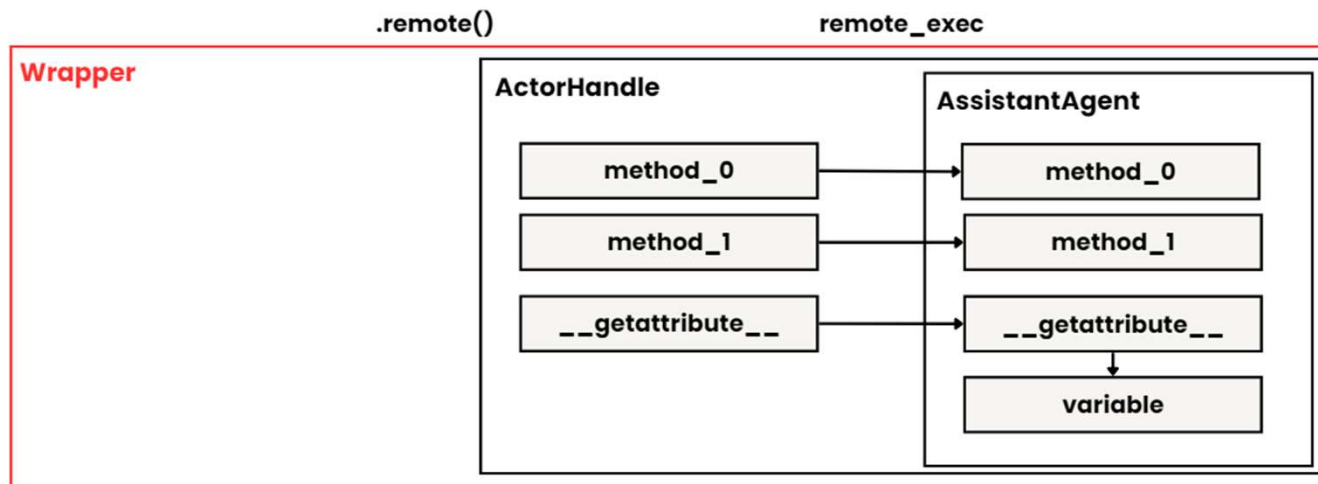
Actor Initialization

```
3 remote_agent_cls = ray.remote(agent_cls)
4 options = {**task_options} if task_options else {}
5 self.remote_agent = remote_agent_cls.options(max_restarts=-1, max_task_retries
=-1, **options).remote(*args, **kwargs)
```



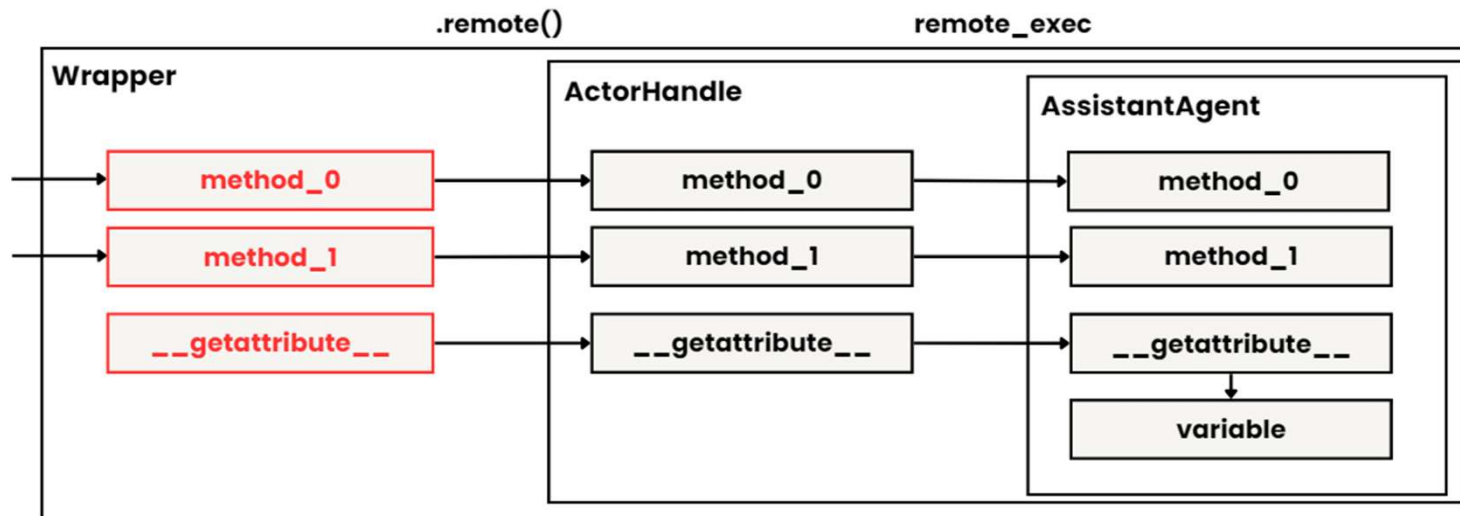
Wrapper Initialization

```
1 class Wrapper():
2     def __init__(self, agent_cls, task_options=None, *args, **kwargs):
3         remote_agent_cls = ray.remote(agent_cls)
4         options = {**task_options} if task_options else {}
5         self.remote_agent = remote_agent_cls.options(max_restarts=-1, max_task_retries
6             =-1, **options).remote(*args, **kwargs)
7         self._add_methods()
```



Method Mapping

```
1 class Wrapper():
2     def __init__(self, agent_cls, task_options=None, *args, **kwargs):
3         remote_agent_cls = ray.remote(agent_cls)
4         options = {**task_options} if task_options else {}
5         self.remote_agent = remote_agent_cls.options(max_restarts=-1, max_task_retries
6             =-1, **options).remote(*args, **kwargs)
7         self._add_methods()
```





Built-in Function in Python

`dir([object])`: is a built-in function in Python that is **used to retrieve the list of attributes and methods of an object**.

`getattr(object, name, default)`: is a built-in Python function **used to retrieve an attribute of an object**. It allows you to access an attribute dynamically.

- **object**: The target object to which you want to access.
- **name**: The name of the attribute you want to get (as a string).
- **default**: An optional value that is returned if the attribute is not found. If not provided, and the attribute doesn't exist, an `AttributeError` is raised.

`setattr(object, name, value)`: is a built-in Python function that **allows you to dynamically set or create an attribute on an object at runtime**.

- **object**: The target object to which you want to add or modify an attribute.
- **name**: The name of the attribute you want to set (as a string).
- **value**: The value you want to assign to the attribute.

With above functions, **wrapper can set the actor's remote call names as its own methods and set the method's values to their corresponding remote calls.**



Method Mapping

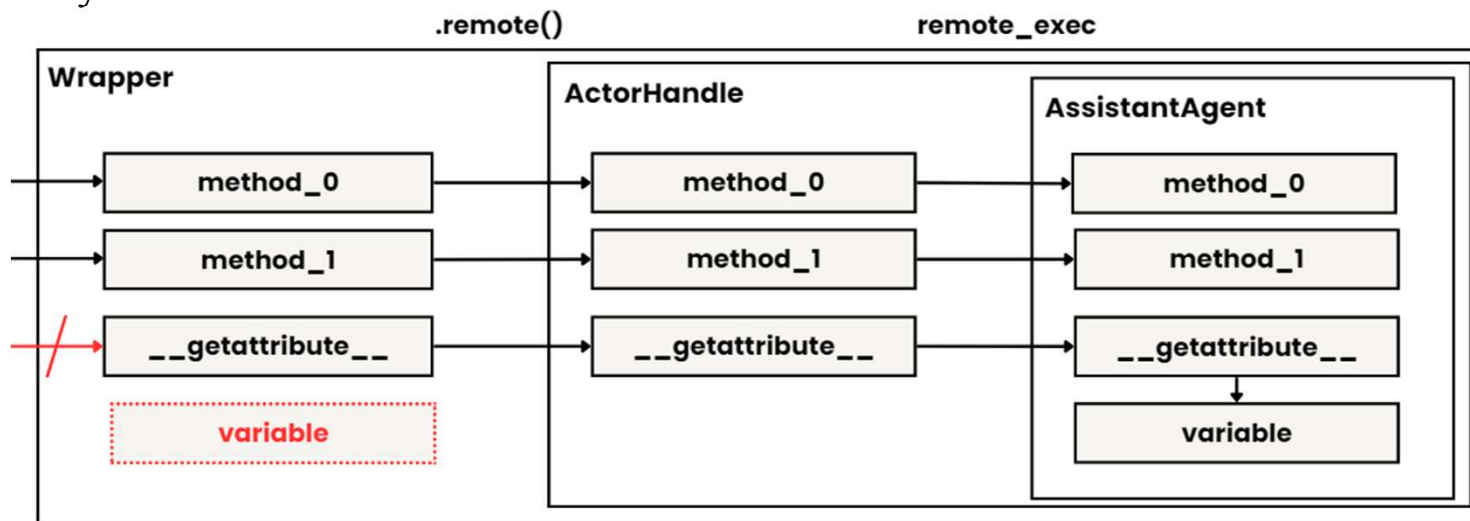
```
8  def _add_methods(self):
9      # Iterate through the attributes of the Actor
10     for name in dir(self.remote_agent):
11         # Get the Actor's method reference by its name
12         attr = getattr(self.remote_agent, name)
13         # Check if the attribute is callable (i.e., a method)
14         if callable(attr):
15             # Set the remote method on the current object with the same name
16             setattr(self, attr._method_name, attr.remote)
```

Fig. Code of Method Mapping

Get Variable

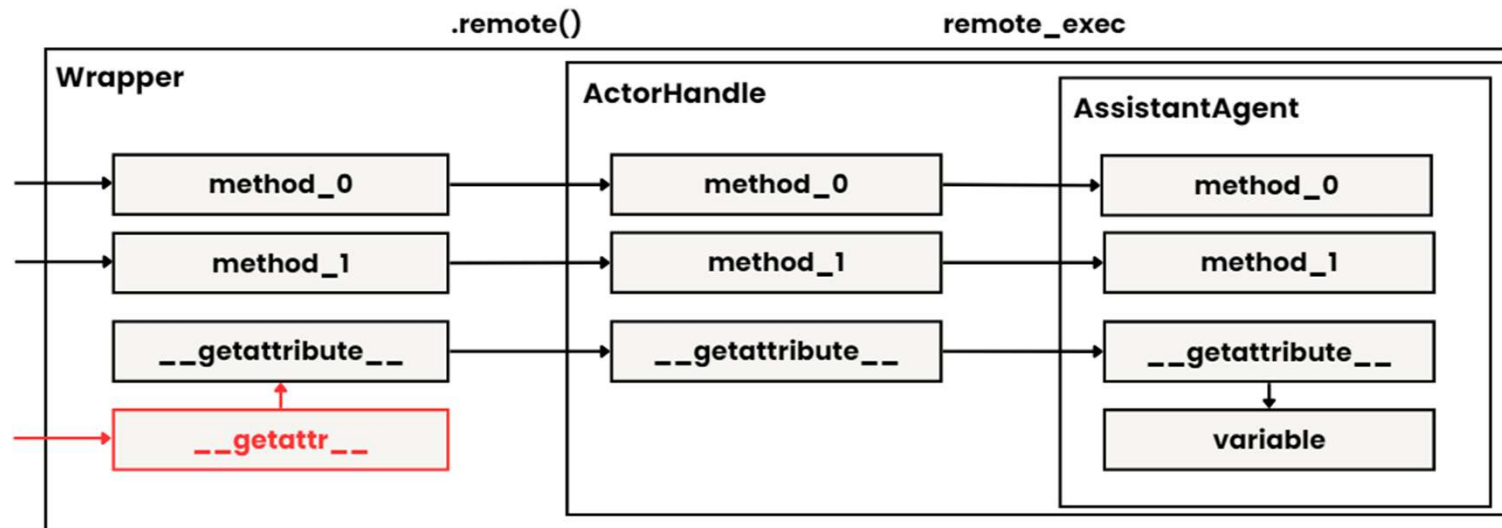
`__getattribute__(self, name)`: In Python, `__getattribute__` is a magic method that is automatically called whenever any attribute of an object is accessed. **If the code in `__getattribute__` doesn't find the attribute, it looks to see if `__getattr__` is defined.** If so, it is called. Otherwise, `AttributeError` is raised.

`__getattr__(self, name)`: In Python, `__getattr__` is a magic method that is **called when an attribute is accessed on an object but is not found** in the object's instance dictionary or its class hierarchy.



Get Variable

```
18     def __getattr__(self, name):
19         try:
20             return ray.get(self.__getattribute__(name))
21         except Exception as e:
22             print(f"Attribute '{name}' does not exist")
```



Wrapper

So far, we have solved the following problems through the Wrapper: **actor instantiation**, **remote method invocation**, and **accessing the actor's attributes**. **The remaining issue to address is determining when to call ray.get(), which will be resolved during the architecture design phase.**

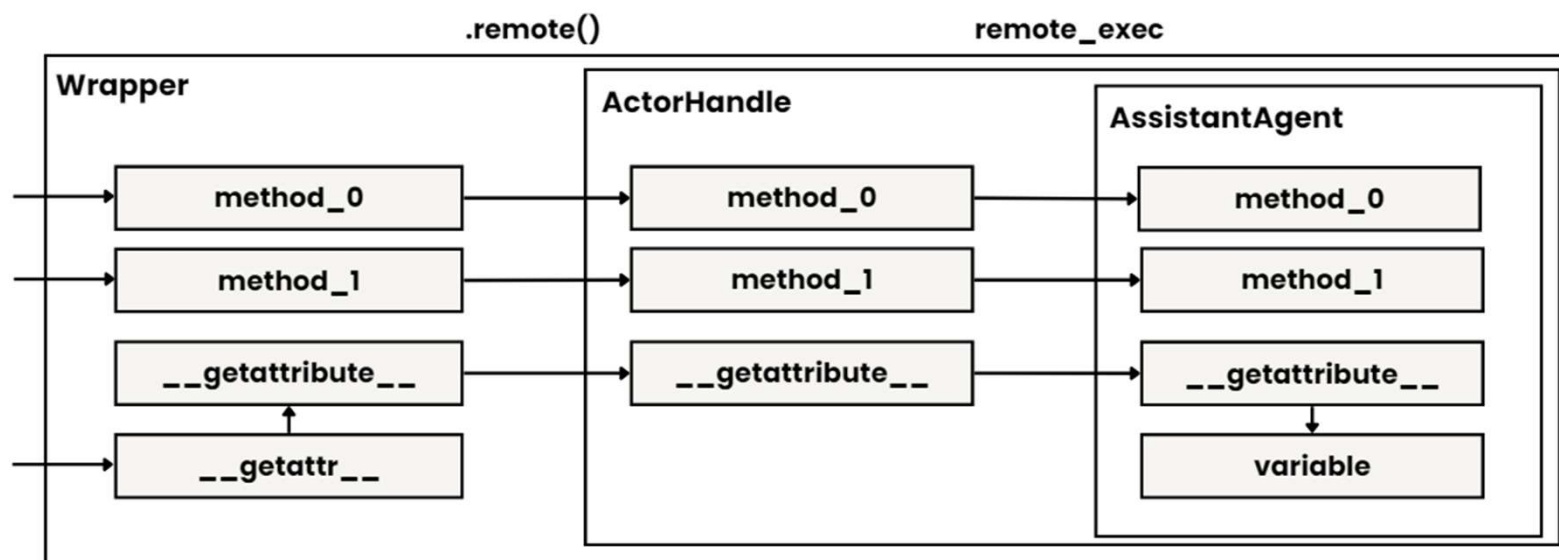


Fig. Wrapper Overview

AutoGen

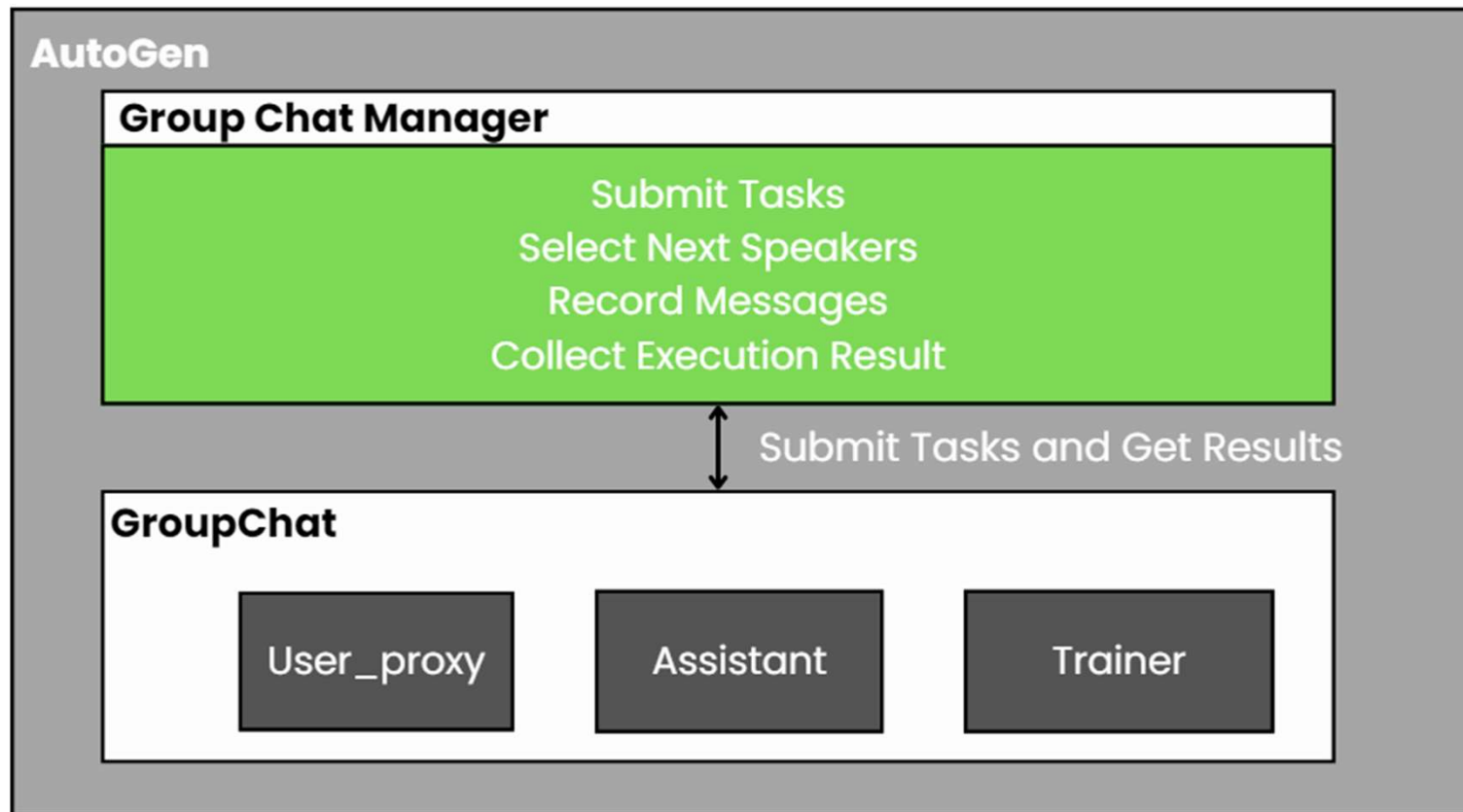


Fig. Architecture of AutoGen

AutoGen with Wrapper Architecture

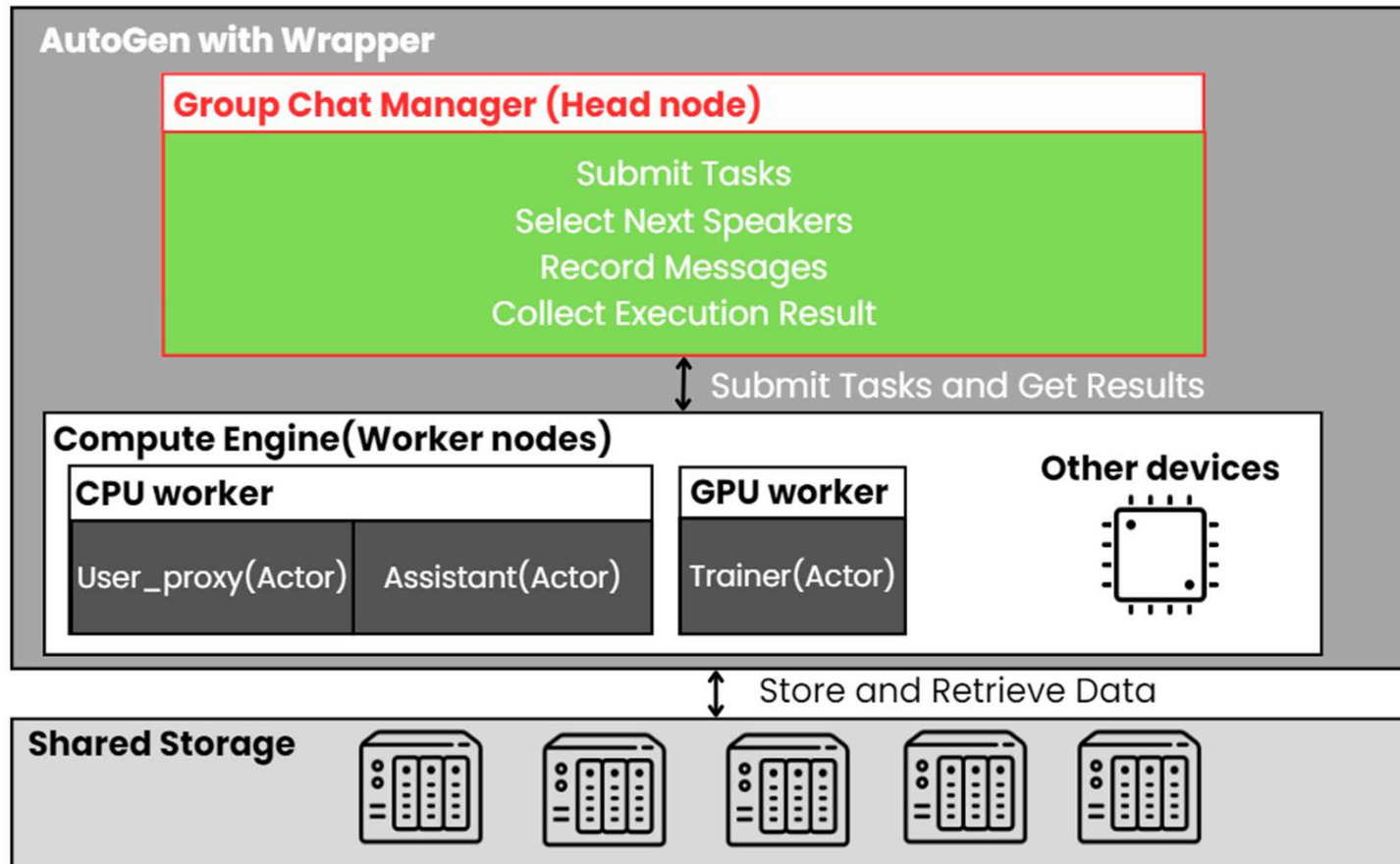


Fig. AutoGen with Wrapper



Challenge of AutoGen with Wrapper

In **AutoGen with Wrapper**, the AutoGen agents have been wrapped into Ray Actors through the wrapper, **enabling the distribution of agents across different hardware for execution.**

However, there are the following two issues:

- **Human intervention:** AutoGen implements manual intervention through the Python **input()** function, where the user proxy agent executes it to obtain user input from the terminal. However, when the user proxy agent is wrapped into an actor and runs on different processes or even different machines, this functionality fails.
- **Blocking task execution:** After the GroupChatManager assigns an agent to speak, it must wait for the reply to be completed before proceeding. If an agent takes a long time to execute, the GroupChatManager's waiting will result in reduced system efficiency.



Human intervention

Human intervention: To address this issue, we adopt a method of centralized management of user input.

- When the user proxy agent requires user input, it no longer directly retrieves it from the terminal, but instead **responds with a specific signal, "GetHumanInput."**
- Once the GroupChatManager receives this response, it will perform the terminal input operation and pass the obtained content back to the agent.
- However, since the GroupChatManager relies on terminal input, it cannot be distributed to other machines as an actor.
- Thus, we have **decoupled the GroupChatManager from the role of an agent and restructured it into a controller**, specifically responsible for managing the overall conversation flow.
- Additionally, in each conversation, we will **launch a new Host agent to take over the tasks of communicating with other agents**, which were previously handled by the GroupChatManager.

AutoGen with Wrapper Architecture

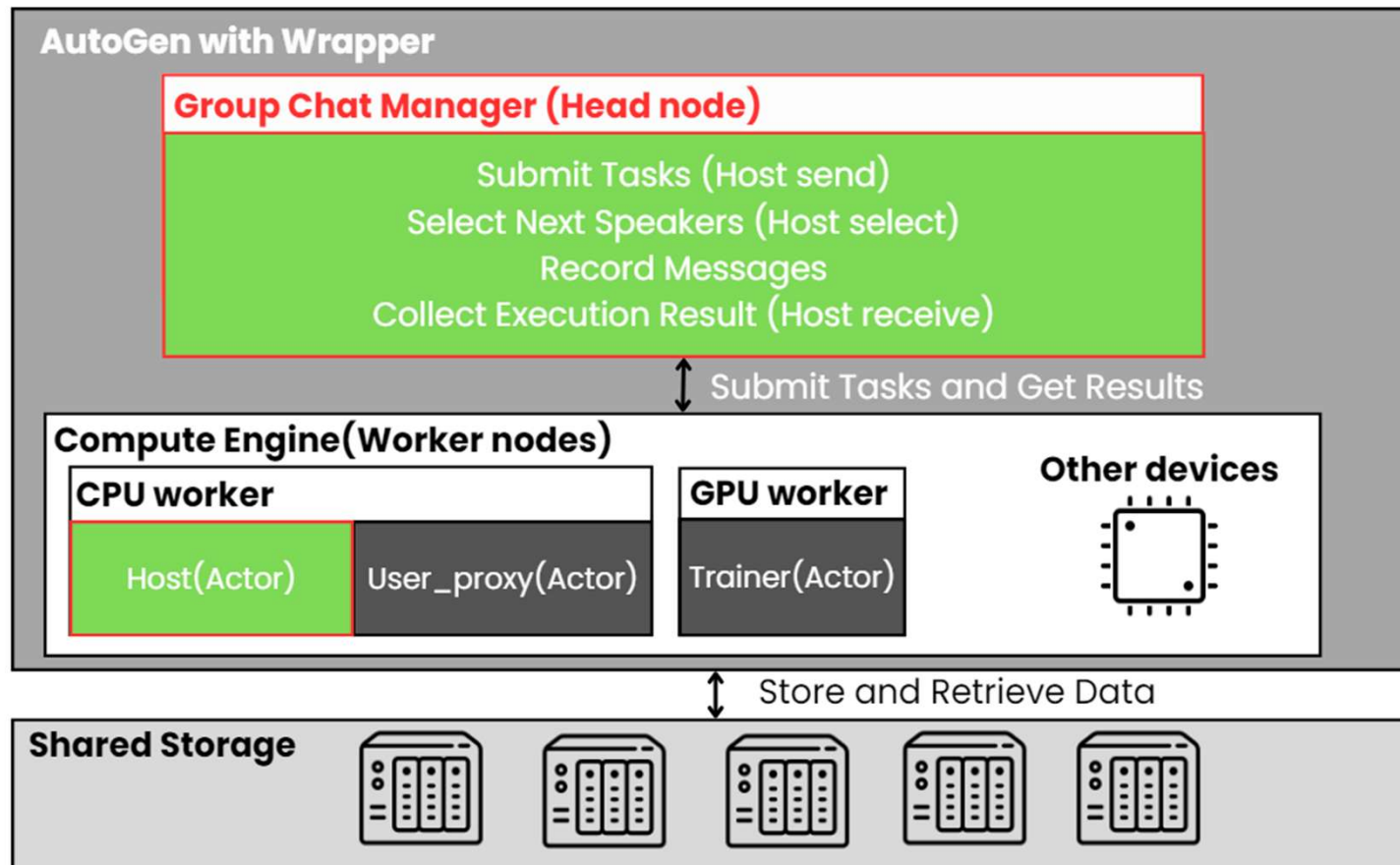


Fig. AutoGen with Wrapper: GroupChatManager turn into controller interact with Host



Blocking task execution

Blocking task execution: Since the **GroupChatManager** in **AutoGen** requires waiting for task results when assigning tasks to agents, causing the main process to be blocked, we addressed this issue and implemented asynchronous execution in **AsyncGen** by **separating the task waiting process from the main process**.

- The original **GroupChatManager** was split into a **ChatCoordinator** and an **AgentSupervisor**.
- The **AgentSupervisor**'s main task is to **monitor whether the agent tasks being executed are completed and place the results into a queue** for the **ChatCoordinator** to use.
- The **ChatCoordinator** is primarily responsible for **coordinating the conversation**. Its duties include:
 - Broadcasting the latest messages to all agents.
 - Selecting the next agent to execute tasks based on the previous round's responses.
 - Recording the entire conversation process.

AsyncGen Architecture

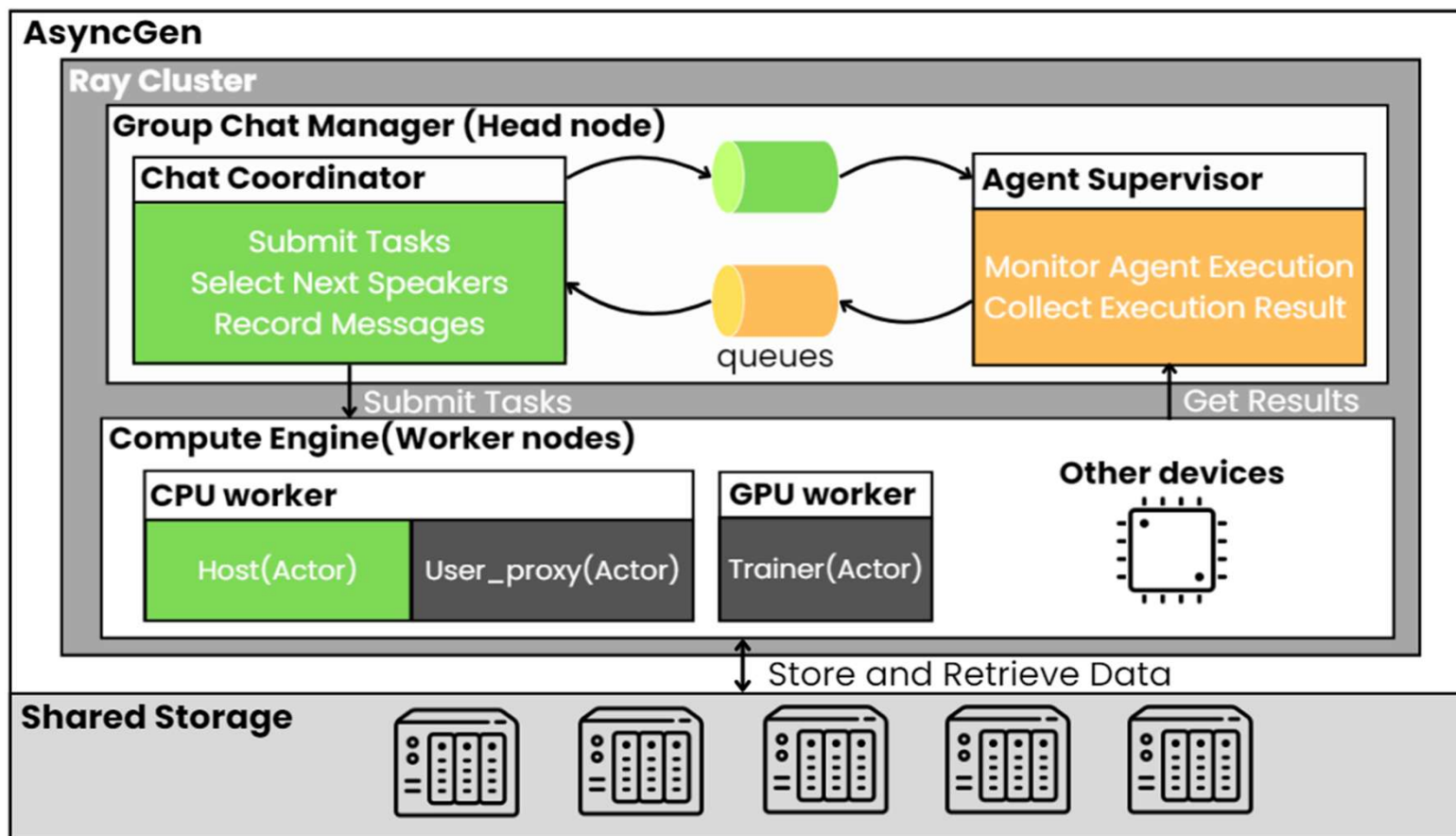


Fig. Architecture of AsyncGen

Workflow

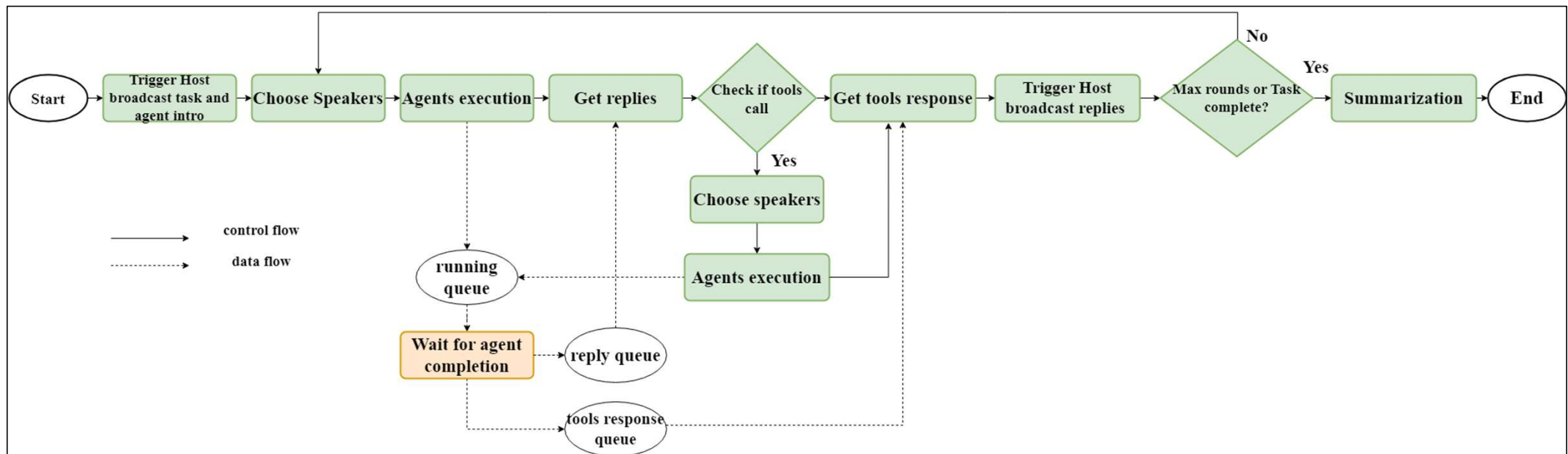


Fig. Workflow of AsyncGen: The ChatCoordinator (green) interacts with the AgentSupervisor (orange).



Implementation of Supporting Features



Parameter Settings

Parameter Settings:

Customizable Parameters for Resource Allocation:

- Different agents have varying hardware requirements depending on their tasks.
- Users can configure system behavior by customizing parameters to allocate agents to suitable machines.

Wrapper integrates parameter configuration, allowing users to pass:

- Agent class
- Initialization parameters (e.g., agent name, purpose)
- Resource requirements

The system automatically assigns agents to appropriate machines based on these parameters.



Backup Mechanisms

Ray's Automatic Actor Restart:

- Users can set `max_restarts=-1` in the options parameter to enable unlimited automatic restarts for actors.
- Default value is 0 (no restarts) if not specified.

Custom Backup and Recovery Mechanism:

- **Issue:** Simple restarts do not restore the agent's pre-crash state or continue interrupted tasks.
- **Solution:**
 - System saves chat history in a user-specified folder before task execution.
 - Upon an agent crash and restart, the system:
 - Checks for backup data in the folder.
 - Restores dialogue history from the backup.
 - Resumes the interrupted task.



Chat History synchronization

In **AutoGen**, all agents independently recorded the full conversation history.

- An agent chat history primarily consists of two types of messages: **messages sent by the Host to agents** (e.g., broadcasts of other agents' messages or initial tasks) and **messages sent by agents to the Host**.

In **AsyncGen**, where operations are asynchronous, new challenges arise:

- Faster agents may complete tasks and send results to the Host, which immediately broadcasts these results to other agents.
- Agents still working on their tasks may miss these broadcasts, leading to gaps in their conversation history and breaking overall consistency.

To address this issue, a **dictionary-based mechanism** is introduced to track each agent's status and any missed messages. The process works as follows:

- If an agent's status is **busy**, the broadcasted message is **stored in agent_status_dict**.
- If an agent's status is **idle**, the Host first send the missed messages stored in agent_status_dict in order. After confirming these messages are received, the current broadcast message is sent.



Experiment



Experiment Overview

This chapter describes the experiments conducted in this study. Specifically, Experiment 1 compares the performance of AsyncGen with AutoGen, while the other experiments focus on evaluating the improved system's capabilities in different scenarios:

- **Experiment 1:** **Evaluates the capability of asynchronous execution**, verifying whether the improved system can enhance execution efficiency for compute-intensive tasks.
- **Experiment 2:** **Tests support for heterogeneous hardware** by examining the impact of adding different hardware resources on task execution efficiency.
- **Experiment 3:** **Tests fault tolerance** by observing task recovery and execution time differences under various failure probabilities.

Experiment Dataset



Colosseum O-RAN COMMAG Dataset

- The dataset is based on a **5G network communication simulation around the Colosseum**.
- It includes data on downlink speeds, uplink speeds, and other related metrics generated from communication between base stations (BS) and surrounding user equipment (UE).
- The dataset contains data from four base stations (BS). Each base station is associated with three CSV files, where each CSV file represents data from a single user equipment (UE). The number of data points in each UE's file varies, ranging from 1900 to 2000 time steps.
- In total, the dataset consists of approximately 22,800 to 24,000 data points across all the UEs.
- We selected **17 metrics** from the dataset to use as features for training the model.

Bonati, Leonardo, et al. "Intelligence and learning in O-RAN for data-driven NextG cellular networks." *IEEE Communications Magazine* 59.10 (2021): 21-27.



Experiment Scenario

Simulate a real **5G downlink speed prediction server**, which includes these agents:

- **Data Collector:** Retrieves user input data (100 time steps per batch) and tracks the number of batches processed since the last fine-tuning. Once a specific number of batches are collected, fine-tuning is triggered for this round.
- **Data Processor:** Preprocesses the input data for either training or inference, ensuring the data is in the correct format for the model.
- **Predictor:** Uses the pre-trained model to predict the downlink speed for the next time step based on processed input data.
- **Tuner:** Updates the model by fine-tuning it using recent data when requested, improving prediction accuracy.
- **Visualizer:** Generates graphs to display historical prediction results, including trends over time or the impact of fine-tuning.
- **Executor Agents:** Includes five executor agents, each handling tasks for the corresponding agent (Data Collector, Data Processor, Predictor, Tuner, Visualizer).
- **CEO:** Oversees the workflow, ensuring all agents perform their tasks correctly and resolving errors when needed.
- **Host:** The host coordinates communication between agents, ensuring smooth information flow for the experiment.

Experiment Setup



Model and Hyperparameters

- Model: 3-layer LSTM
- Epochs: 100
- Optimizer: Adam
- Learning Rate: 0.005
- Loss Function: Huber Loss
- Huber Delta: 0.3



Experiment1

Experiment1

- The experiment compares the speed differences between sequential execution in AutoGen and parallel execution in AsyncGen on a single machine, aiming to verify that the improved system's asynchronous execution capability can enhance task execution efficiency.
- In this experiment, **only a single machine with GPU will be used**, operating in the 5G base station downlink prediction scenario.
- Data Collector will perform **20 rounds of data collection**, gathering **100 data points in each round** and carrying out corresponding data preprocessing, model inference, and result visualization.
- Additionally, the **model will be fine-tuned each time 500 data points** are collected.
- The experiment will end once all the data has been processed.

Experiment1 AutoGen Workflow

AutoGen

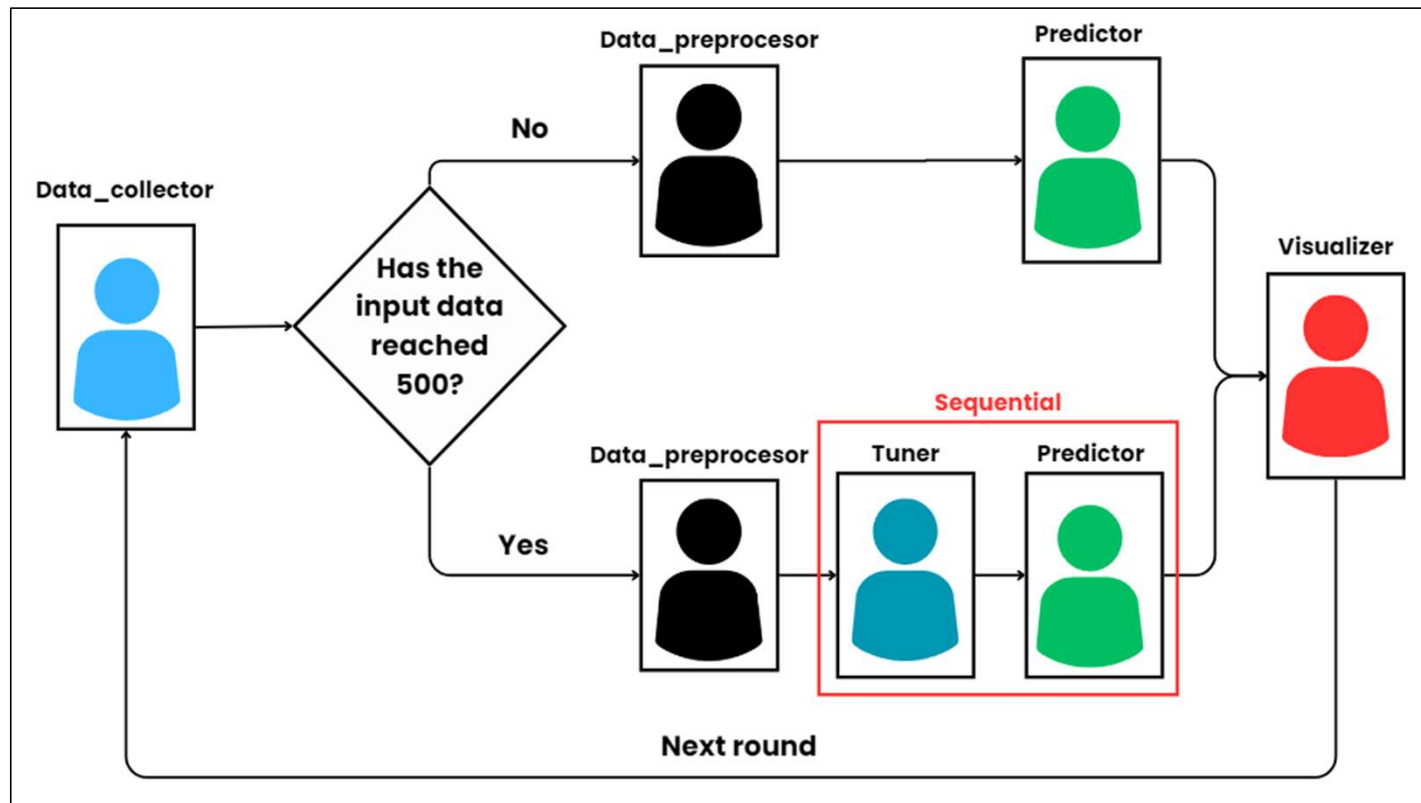


Fig. AutoGen Sequential Workflow in Experiment1

Experiment1 AsyncGen Workflow

AsyncGen

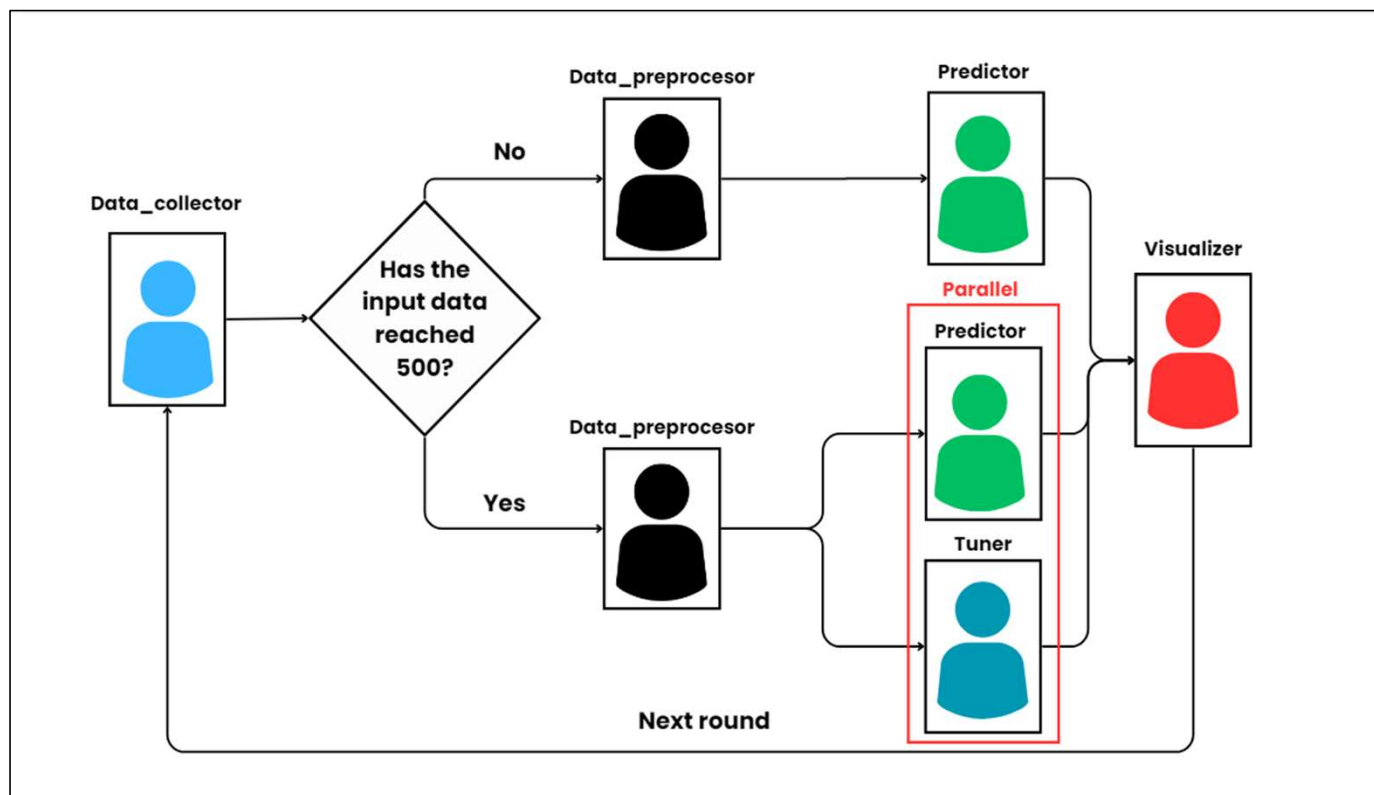
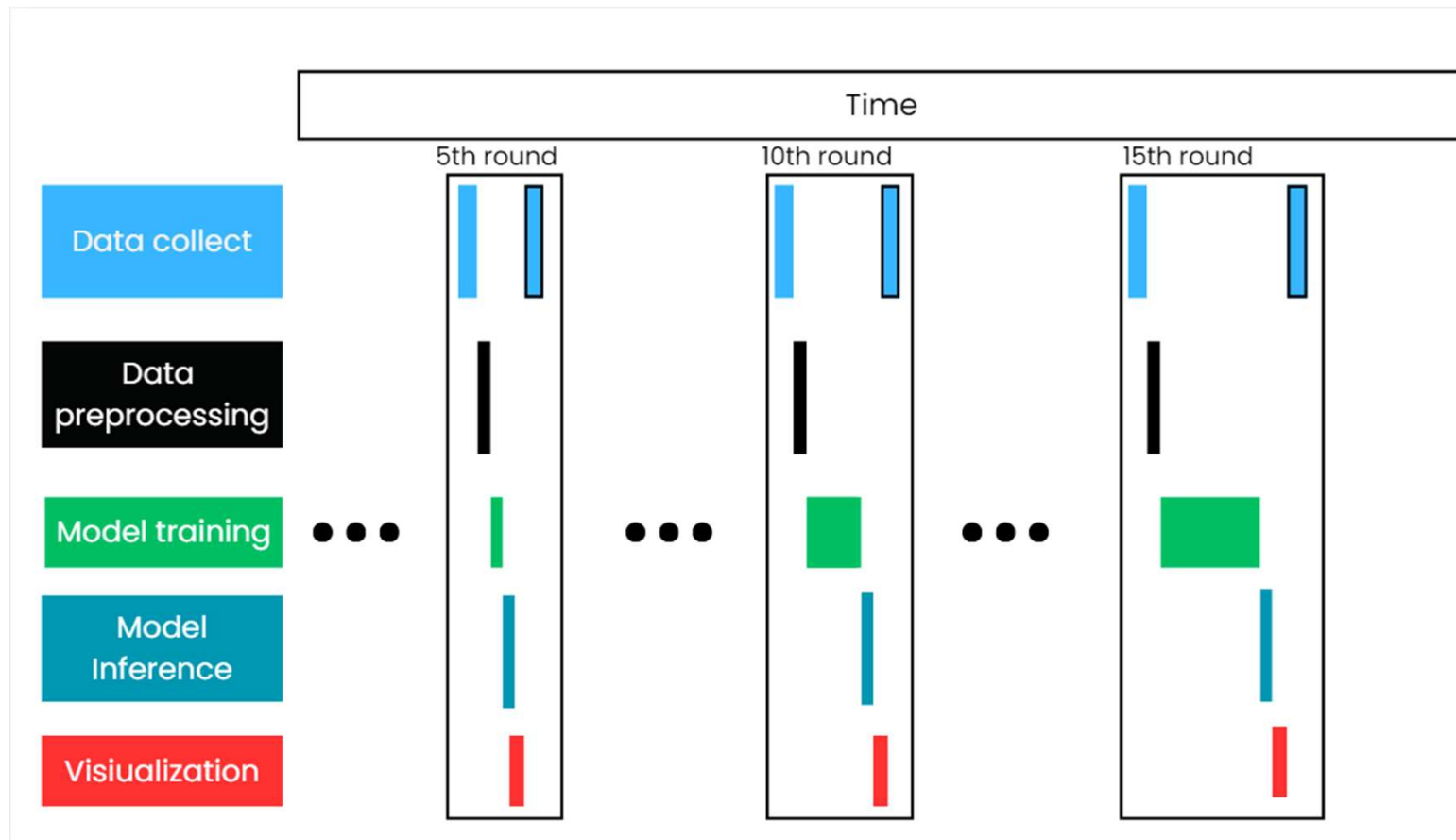


Fig. AsyncGen Parallel Workflow in Experiment1

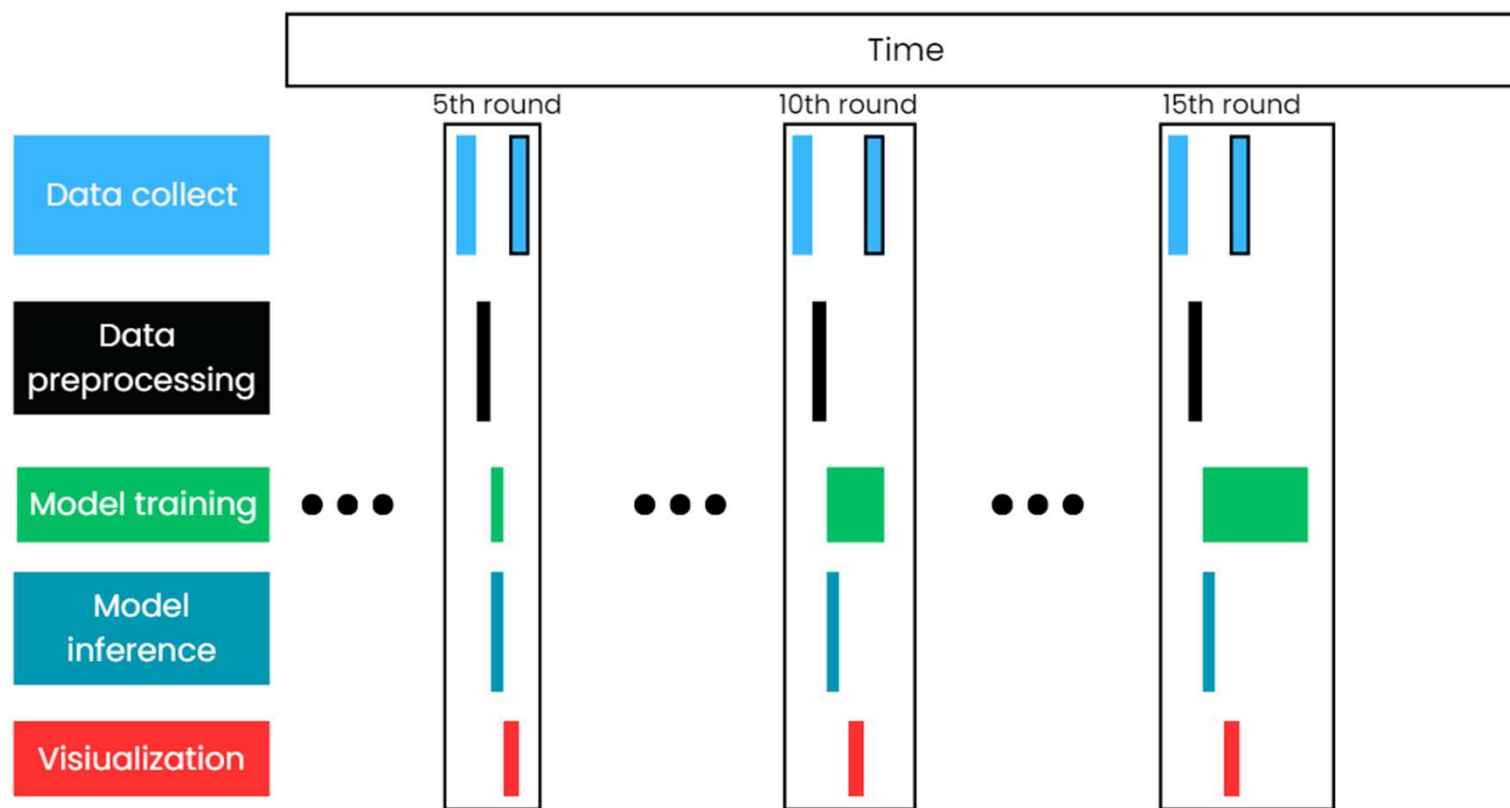
Experiment 1 AutoGen Workflow

AutoGen (sequential)



Experiment1 AsyncGen Workflow

AsyncGen (parallel)



Experiment1 Result

- **AutoGen:** 29m 35s
- **AsyncGen:** 19m 30s

By running the Tuner asynchronously, AsyncGen reduces execution time by approximately 34% compared to AutoGen, demonstrating a significant improvement in efficiency.

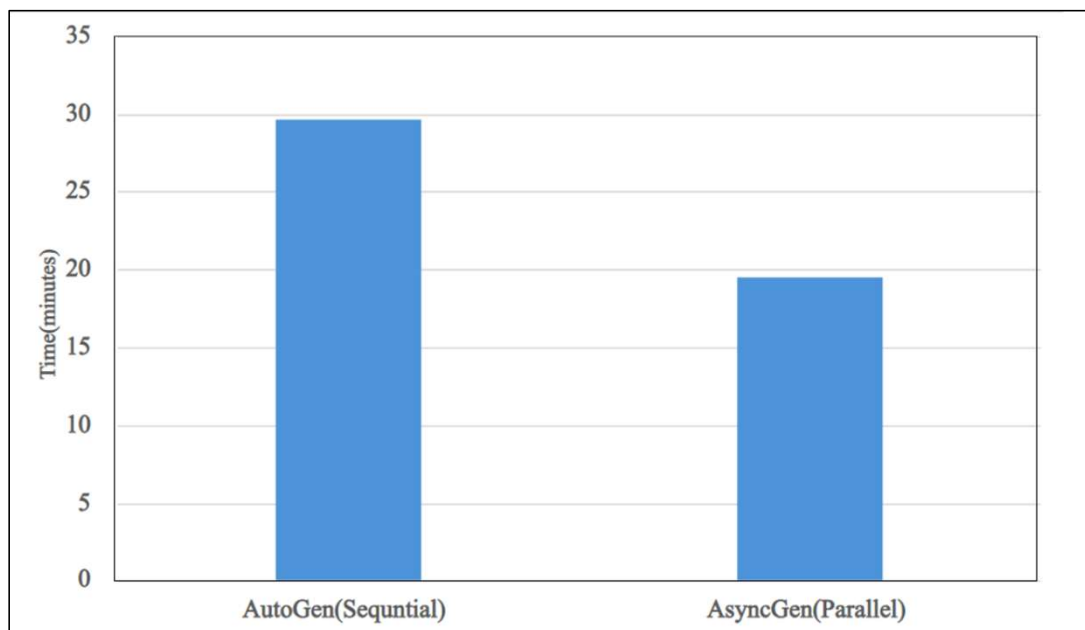
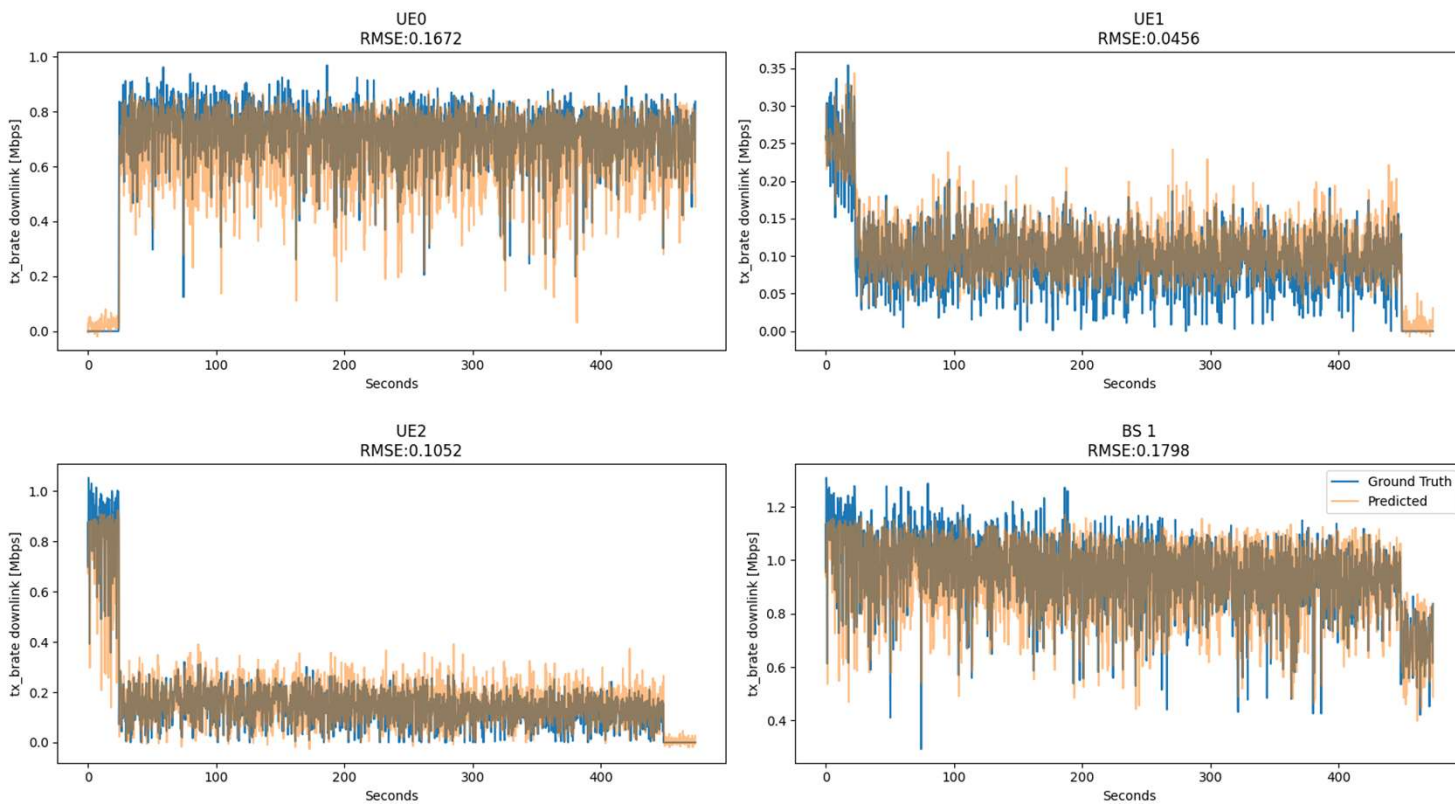


Fig. AutoGen and AsyncGen performance comparison

Experiment1 Result

Inference Result





Experiment2

Experiment2

- The purpose of Experiment 2 is to verify whether the improved system has **scalability**.
- In this experiment, the user has three computers: one with only **CPU(M1)**, one with **GPU(M2)**, and one using **Network File System (NFS) as shared storage(M3)**.
- In the **initial phase** of the experiment, the user runs tasks using **only the CPU**.
- Then, **M2 (the machine with GPU) is added to the ray cluster** to run the experiment and compare the execution time with GPU assistance.
- Unlike Experiment 1, in this experiment, all the datasets are collected in one round of conversation, and only one model training is performed.

Experiment2 Result

- **Only M1:** 1h 14m 21s
- **M1+M2+M3:** 8m 35s
- **Only M2:** 5m 0s

This experiment shows that adding hardware resources (like GPU, TPU, etc.) to the Ray cluster and configuring agents' hardware requirements in AsyncGen can improve efficiency and reduce execution time.

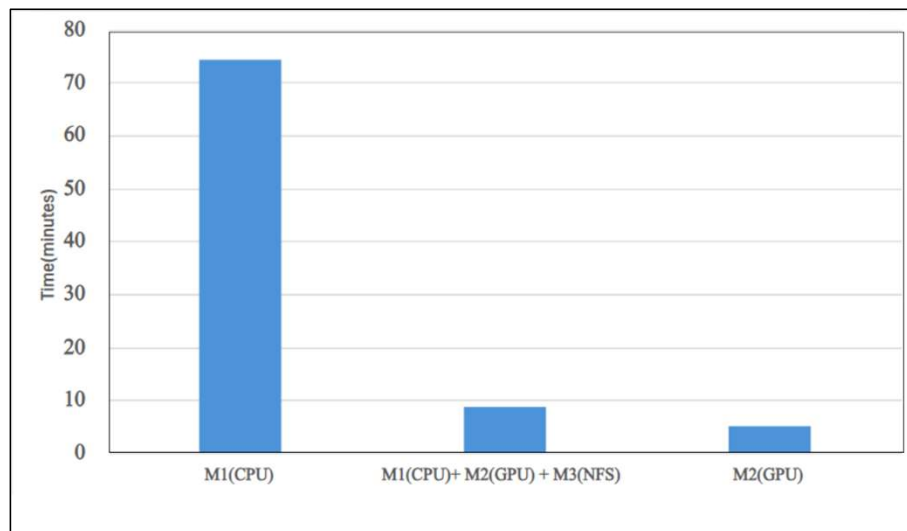


Fig. Execution Time Comparison: M1 vs M1+M2+M3 vs M2



Experiment3

Experiment3

- The purpose of Experiment 3 is to verify whether the improved system has fault tolerance.
- In this experiment, the user has **M1, which is equipped with both CPU and GPU**, and M3, which uses **NFS as the storage system**.
- To prevent the system from crashing due to a single error, the user **sets M3 as a backup location** and requires the agent to store conversation logs in M3 during the execution process.
- When necessary, the system can retrieve backup information from M3 and restore the execution state.

Experiment3 Result

- As shown in the figure, the probability of agent crashes increases from left to right, as follows: no crash, 10% crash probability per execution, 20%, 40%, and 60% crash probability.
- In each trial of Experiment 3, **the tasks were successfully completed**, but as the number of crashes increased, the execution time also increased.

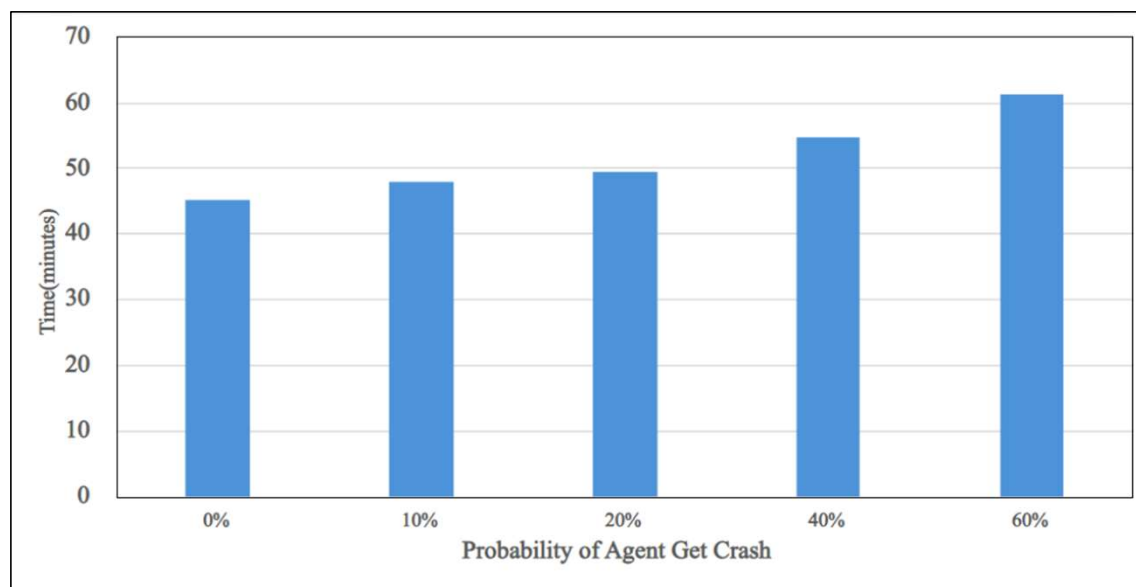


Fig. The Relationship Between Failure Probability and Execution Time



Experiment3 Result

- Metrics are shown in the table. It presents the **total number of crashes**, **average recovery time**, and **average crash interval time** (i.e., the average time between two consecutive crashes) under different crash probabilities.
- It can be observed that the number of crashes increases with the rise in crash probability; the **average recovery time** remains stable at around **two seconds**; and the average crash interval time shortens as the crash probability increases.

Table: Comparison of Results with Different Crash Probabilities

Crash Probability (%)	Total Crashes	AVG Recovery Time (s)	AVG Time Between Crashes (s)
10	35	1.95	476.23
20	78	2.05	318.21
40	249	2.00	113.86
60	540	1.91	66.35



Conclusion

- We presents an asynchronous distributed multi-agent conversation framework, AsyncGen, which integrates AutoGen with Ray.
- By mapping AutoGen's agents to Ray's Actors and improving the system architecture and workflow, dynamic resource scheduling and fault tolerance are achieved, overcoming the limitations of the original system that only supported single-machine operation.
- Experimental results demonstrate that this framework offers significant advantages in **asynchronous execution** and effectively enhances the system's **fault tolerance** and **scalability**.



Limitation & Future Work

Although the proposed asynchronous multi-agent framework combining AutoGen and Ray showed significant benefits in fault tolerance, scalability, and efficiency, there are still **limitations** that require further optimization.

- **Limitations of the transition dictionary:** The original transition dictionary mechanism is incompatible with the asynchronous execution model. Future improvements could involve dynamic updates to the dictionary or adding a state management system for better multi-agent decision-making.
- **Underutilization of distributed systems:** The framework retained the centralized communication model, limiting the benefits of distributed systems. Adding an agent discovery mechanism could allow agents to dynamically choose collaborators based on task needs.
- **Manual workflow design:** Dependencies between tasks require users to manually design workflows, adding complexity. Automating this process could reduce user burden and improve efficiency.



The End

Thank you for listening!